Proyecto 2: Juego en Assembly sobre Pochoco SoC

Juego de reflejos en RV32E con assembler propio

Arquitectura de Computadores
Universidad de los Andes
Semestre 2026-2

Integrantes: Nicolás Araneda Vicente Calderón Santiago Muñoz

Octubre de 2026

Todo el código está en https://github.com/Vcalderone/Proyecto2_Arquitectura_De_Computadores. Este documento usa los mismos nombres de módulo, etiqueta y constante que el repositorio.

1 Arquitectura del juego, contador de ciclos y temporización

1.1 Arquitectura

El diseño parte de `nic0villegasc/pochoco_soc`, que aporta el Espino Core (RV32E), una RAM de 512 palabras compartida entre instrucciones y datos, y el bloque de periféricos. El top propio, `rtl/game_top.v`, instancia `pochoco_soc` con `MemFile = "sw/game.hex"` y expone los pines de la Go Board definidos en `goboard.pcf`. El programa ocupa 115 de las 512 palabras y queda dentro del bitstream, porque la RAM se inicializa con `$readmemh` al sintetizar.

La RAM parte en `0x0000_0000`, que es también la dirección de arranque del core, y los periféricos en `0x8000_0000` (bit 31 de la dirección en uno). La Tabla 1 muestra los registros que usa el programa.

Offset	Nombre	Acceso	Bits	Función
`0x00`	DISP	Escritura	`[7:0]`	Dos dígitos del display, un nibble por dígito
`0x04`	LEDS	Escritura	`[3:0]`	Los cuatro LEDs
`0x08`	BTN	Lectura	`[3:0]`	Los cuatro botones, sincronizados y con debounce
`0x0C`	CYCLES	Lectura	`[31:0]`	Contador libre de ciclos de reloj

Tabla 1: Registros de periférico, con offsets desde `0x8000_0000`. Fuente: `docs/memory_map.md`.

El display decodifica cada nibble en hexadecimal, así que para mostrar el número 37 hay que escribir `0x37` y no el decimal 37 (`0x25`). Por eso el programa convierte a BCD antes de escribir, en la subrutina `show_bcd`.

1.2 Modificaciones al RTL

La Tabla 2 resume los cambios respecto del repositorio base. El core y la RAM no se modificaron.

Archivo	Cambio	Motivo
`pochoco_periph.v`	Contador CYCLES de 32 bits en el offset `0x0C`	Contador de ciclos accesible por el programa, que pide el enunciado
`pochoco_periph.v`	Sincronizador de dos flip-flops y debounce entre `i_Switch` y BTN	Los botones son asíncronos y rebotan
`debounce.v`	Archivo nuevo con el debounce de los cuatro botones	Mismo esquema del Proyecto 1: un cambio se acepta tras tres muestras iguales, una cada 2^{15} ciclos (1,31 ms)
`pochoco_soc.v`	Reset de encendido de 16 a 4096 ciclos (164 µs)	En la placa, con 16 ciclos el core partía antes de que la RAM entregara datos válidos
`pochoco_soc.v`	El esclavo SPI deja de instanciarse	El juego no lo usa y ocupa unas 86 celdas lógicas; sus pines quedan con valor fijo
`game_top.v`	Archivo nuevo	Top de síntesis pedido por el enunciado

Tabla 2: Modificaciones al RTL respecto del repositorio base.

Con estos cambios el diseño usa 1137 de 1280 celdas lógicas (88 %) y 12 de 16 bloques de RAM de la iCE40 HX1K, con una frecuencia máxima de 44,5 MHz según `nextpnr-ice40`, sobre los 25 MHz del reloj de la placa.

1.3 Unidad de tiempo y conversión desde ciclos

El reloj es el oscilador de 25 MHz de la Go Board, sin PLL. CYCLES suma 1 en cada ciclo desde el reset y el software no lo puede detener ni reiniciar, así que toda duración se mide como la diferencia entre dos lecturas. La resta es de 32 bits y sale correcta aunque el contador haya dado la vuelta entre ambas lecturas (lo hace cada $2^{32}/25\,\text{MHz} \approx 172\,\text{s}$), y por eso las comparaciones de duración usan `bltu`, sin signo.

La unidad de tiempo del juego es la décima de segundo. Una décima equivale a

$$\text{CYCLES_PER_TENTH} = \frac{f_{clk}}{10} = \frac{25\,000\,000}{10} = 2\,500\,000 \text{ ciclos},$$

y una duración medida en décimas es

$$t = \left\lfloor \frac{\text{CYCLES}_1 - \text{CYCLES}_0}{2\,500\,000} \right\rfloor .$$

El core no tiene multiplicación ni división, así que `cyc2tenths` divide por restas repetidas y `wait_tenths` pasa de décimas a ciclos por sumas repetidas. `CLK_HZ` y `CYCLES_PER_TENTH` son dos literales separados en `game.s` porque el assembler no evalúa expresiones: si cambia la frecuencia, hay que cambiar ambos.

Se dejó un solo contador de ciclos, sin un contador de décimas en hardware. Un contador de décimas libre está en una fase cualquiera cuando el programa lo lee, así que la misma duración real se lee a veces como N y a veces como $N+1$ décimas. Midiendo en ciclos el error de cuantización baja a un ciclo (40 ns).

1.4 Temporización de los tres segundos

La espera al inicio de cada ronda es `ESPERA_TENTHS = 30`. `wait_tenths` la convierte primero a $30 \times 2\,500\,000 = 75\,000\,000$ ciclos, toma una lectura de CYCLES y repite lecturas hasta que la diferencia llega a ese total:

```
    lw   x9, OFF_CYCLES(x2)    # marca inicial
wt_wait_loop:
    lw   x13, OFF_CYCLES(x2)   # marca actual
    sub  x14, x13, x9
    bltu x14, x11, wt_wait_loop  # x11 = total en ciclos
```

La salida del lazo ocurre en la primera lectura con una diferencia mayor o igual al total, así que la espera dura 3,0 s o unos pocos ciclos más, nunca menos, como pide el *al menos tres segundos* del enunciado.

1.5 Máquina de estados

La Tabla 3 resume el programa. Los nombres son las etiquetas de `sw/game.s`.

Estado	Salida	Qué hace
`_start`	–	Fija la base de periféricos, el polinomio del LFSR, aciertos y suma en cero
`start_screen`	`00`, LEDs apagados	Espera una pulsación, siembra el LFSR con CYCLES y espera a que se suelte
`round_begin`	Cuatro LEDs, número de ronda	Espera `ESPERA_TENTHS` décimas
`arm`	Solo el LED objetivo	Espera que no haya botones apretados, sortea el objetivo y toma t_0
`wait_press`	–	Lee BTN en un lazo y avanza el LFSR en cada vuelta
`miss`	`EE`, LEDs apagados	Botón distinto del objetivo: espera `ERROR_TENTHS` y vuelve a `round_begin`
`hit`	Tiempo en décimas	Mide, suma, incrementa aciertos y muestra el tiempo `MOSTRAR_TENTHS` décimas
`finish`	`AA`, luego el promedio	Calcula el promedio, hace parpadear los LEDs y una pulsación vuelve a `_start`

Tabla 3: Estados del programa del juego.

Hay una pantalla de inicio porque el LFSR se siembra con el instante de una pulsación del jugador; sin ella el LED de la primera ronda sería el mismo en cada encendido. Un error reinicia la ronda completa, incluidos los tres segundos, y no modifica los aciertos ni la suma. La comparación del botón es sobre la máscara completa de cuatro bits, así que cualquier valor distinto del LED objetivo cuenta como error.

2 Assembly documentado, medición de tiempo y promedio

2.1 Organización del programa

RV32E tiene 16 registros. El programa no usa stack: las subrutinas no son recursivas, así que cada nivel de llamada tiene su propio registro de retorno, `x1` para `show_bcd` y `wait_tenths` y `x15` para `div` y `cyc2tenths`. Ninguna subrutina escribe `x2` a `x8`, por lo que el estado del juego se conserva entre llamadas sin guardarlo en memoria (Tabla 4).

Registro	Contenido	Registro	Contenido
x1	Retorno de nivel 1	x6	Suma de tiempos en décimas
x2	Base de periféricos 0x80000000	x7	Máscara del LED objetivo
x3	Estado del LFSR	x8	t_0 de la ronda, en ciclos
x4	Polinomio del LFSR	x9–x14	Temporales
x5	Aciertos	x15	Retorno de nivel 2

Tabla 4: Asignación de registros en `game.s`.

Las subrutinas son `div` (división entera sin signo por restas, con el divisor en `x12`), `cyc2tenths` (ciclos a décimas), `show_bcd` (escribe un valor de 0 a 99 en el display como dos dígitos decimales) y `wait_tenths` (espera un número de décimas).

2.2 Generación pseudoaleatoria

La ALU del Espino Core ejecuta los desplazamientos como sumas, así que se usa un LFSR de Galois que desplaza hacia la izquierda: el bit que sale es el bit 31, que es el signo del registro, y desplazar un lugar es sumar el registro consigo mismo.

```
wait_press:
    lw   x9, OFF_BTN(x2)
    blt  x3, x0, wp_feedback   # bit 31 de x3 en uno
    add  x3, x3, x3            # x3 <- x3 << 1
    j    wp_lfsr_done
wp_feedback:
    add  x3, x3, x3
    xor  x3, x3, x4            # x3 <- x3 ^ POLY
```

El polinomio `POLY = 0x04C11DB7` es primitivo de grado 32, con período $2^{32} - 1$. La semilla es el valor de CYCLES en el instante de la pulsación de inicio (si es cero se fuerza a uno, porque cero es un punto fijo del LFSR), y después el estado solo avanza. El paso del LFSR está dentro del lazo que espera la pulsación, que da una vuelta cada pocos ciclos, así que el estado avanza cientos de miles de pasos por ronda y depende de cada tiempo de reacción del jugador. El LED objetivo son los dos bits bajos del estado al armar la ronda, y su máscara se arma duplicando un uno esa cantidad de veces.

2.3 Medición del tiempo de respuesta

La medición parte en `arm`, inmediatamente después de encender el LED objetivo, y termina en `hit`, apenas se lee el botón correcto:

```
arm_shift_done:
    sw   x7, OFF_LEDS(x2)     # enciende solo el LED objetivo
    lw   x8, OFF_CYCLES(x2)   # t0 en ciclos
...
hit:
    lw   x9, OFF_CYCLES(x2)
    sub  x10, x9, x8          # duracion en ciclos
    jal  x15, cyc2tenths      # x10 = decimas
```

Se interpretó que el *mismo instante lógico en que comienza la ronda* es el encendido del LED objetivo, porque es desde ahí que el jugador puede reaccionar. Entre la escritura a LEDS y la lectura de t_0 pasan unos tres ciclos (120 ns), despreciables frente a una décima. `cyc2tenths` trunca hacia abajo y satura en `MAX_TENTHS = 99`, el máximo de dos dígitos, y el mismo corte limita su lazo a 99 vueltas. Al tiempo medido se suma el retardo del debounce, de 4 a 5 ms.

2.4 Promedio de las diez rondas

Al completar `RONDAS` aciertos, `finish` muestra `AA` durante `PROMEDIO_TENTHS` décimas, para distinguir el promedio del tiempo de la última ronda, y después el promedio con los LEDs parpadeando:

```
    mv   x10, x6              # suma en decimas
    li   x12, RONDAS          # divisor
    jal  x15, div             # x10 = suma / RONDAS
    jal  x1, show_bcd
```

El promedio queda en décimas, truncado. El divisor es `RONDAS` y no un 10 fijo, porque en la evaluación se pide cambiar una constante del juego: con un 10 fijo, cambiar `RONDAS` terminaría el juego en otra ronda pero seguiría dividiendo por 10. `show_bcd` usa la misma `div` con divisor 10.

2.5 Verificación en simulación

`tb/game_tb.v` recorre una partida completa con un error deliberado. Inyecta un tiempo de reacción conocido de 12 décimas y exige que el display muestre `12` en cada ronda y como promedio. La misma prueba se corre con `RONDAS = 4`, donde el promedio también debe dar 12 (con un divisor fijo en 10 daría $48/10 = 4$). Para que la simulación dure segundos, el programa se ensambla con `-D CYCLES_PER_TENTH=2500`. `tb/game_top_tb.v` prueba los botones con rebotes inyectados y el avance de CYCLES. Todo se corre con `make check`.

3 Assembler: codificación, etiquetas, validación y pruebas

El assembler está en `assembler/`, en Python 3 sin dependencias externas, y no invoca ningún assembler RISC-V. Genera un archivo con una palabra de 32 bits en hexadecimal por línea:

```
python3 assembler/asm.py sw/game.s -o sw/game.hex [-D NOMBRE=VALOR ...]
```

`make asm` hace lo mismo. La opción `-D` reemplaza el valor de un `.equ` sin editar el archivo.

3.1 Sintaxis aceptada

Una instrucción por línea, operandos separados por comas y comentarios desde `#` hasta el fin de línea. Las etiquetas son `nombre:` y pueden ir solas en una línea o antes de una instrucción. Los accesos a memoria usan `desplazamiento(registro)`. Los literales pueden ser decimales, hexadecimales (`0x`) o binarios (`0b`), con signo opcional. Mnemónicos, registros y directivas no distinguen mayúsculas; etiquetas y constantes sí.

Las directivas son `.equ NOMBRE, valor`, que define una constante, y `.word`, que emite palabras y acepta etiquetas. `.section`, `.text` y `.global` se aceptan y se ignoran. Los registros van de `x0` a `x15`, también con sus nombres ABI (`zero`, `ra`, `sp`, `a0`, etc.). Un registro `x16` o mayor es error, aunque cabría en los cinco bits del campo, porque RV32E no lo tiene.

3.2 Instrucciones que usa el juego

El assembler codifica las instrucciones de RV32I en los formatos R, I, S, B, U y J, sin `ecall`, `ebreak`, `fence` ni CSR. La Tabla 5 muestra las que usa `game.s`.

Formato	Instrucciones
R	`add`, `sub`, `xor`
I	`addi`, `andi`, `xori`, `lw`, `jalr`
S	`sw`
B	`beq`, `bne`, `blt`, `bltu`
U	`lui`
J	`jal`
Pseudo	`li`, `mv`, `j`

Tabla 5: Instrucciones usadas por `sw/game.s`.

Los desplazamientos (`sll`, `srl`, `sra` y sus versiones con inmediato) se rechazan salvo con la opción `-allow-shifts`, porque el core los ejecuta como suma y el programa correría mal sin ningún aviso.

3.3 Tratamiento de etiquetas e inmediatos

El assembler recorre el archivo dos veces. En la primera pasada lleva la dirección actual desde cero y guarda cada etiqueta con su dirección y cada `.equ` con su valor; como `li` puede ocupar una o dos palabras, se calcula su tamaño en esta pasada. En la segunda pasada codifica cada línea con la tabla de símbolos completa, lo que permite saltar a una etiqueta definida más abajo. En saltos y bifurcaciones el inmediato codificado es la dirección de la etiqueta menos la dirección de la instrucción.

Los rangos son de -2048 a 2047 para los formatos I y S, desplazamientos pares de -4096 a 4094 bytes para las bifurcaciones, pares de $\pm 2^{20}$ bytes para `jal`, y 20 bits sin signo para `lui`. Un valor fuera de rango es error.

Expansión de `li`. Si el valor cabe en 12 bits con signo se emite un `addi`; si sus 12 bits bajos son cero, un `lui`; si no, `lui` seguido de `addi`. En este último caso, si el bit 11 del valor es uno, `addi` lo interpreta como un inmediato negativo y resta 4096. Por eso la parte alta se redondea:

$$hi = \left\lfloor \frac{valor + 0\text{x}800}{4096} \right\rfloor, \qquad lo = valor - 4096\, hi.$$

Para `li x4, 0x04C11DB7` (el polinomio del LFSR) los 12 bits bajos son `0xDB7`, con el bit 11 en uno, así que $hi = 0\text{x}04\text{C}12$ y $lo = -585$. Se emiten `lui x4, 0x04C12` y `addi x4, x4, -585`, que en `sw/game.hex` son `04c12237` y `db720213`.

3.4 Ejemplo de traducción a código máquina

Con `.equ OFF_LEDS, 4`, la instrucción `sw x9, OFF_LEDS(x2)` de `round_begin` escribe `x9` en la dirección `x2 + 4 = 0x8000_0004`, el registro LEDS. `sw` es de formato S, con el inmediato de 12 bits dividido en dos campos (Tabla 6).

Campo	Bits	Valor	Binario
`imm[11:5]`	31–25	4, bits 11 a 5	0000000
`rs2`	24–20	`x9`, dato	01001
`rs1`	19–15	`x2`, base	00010
`funct3`	14–12	`sw`	010
`imm[4:0]`	11–7	4, bits 4 a 0	00100
`opcode`	6–0	store	0100011

Tabla 6: Codificación de `sw x9, OFF_LEDS(x2)`.

```
0000000 01001 00010 010 00100 0100011
0000 0000 1001 0001 0010 0010 0010 0011  =  0x00912223
```

La palabra `00912223` está en la línea 18 de `sw/game.hex`.

3.5 Validación y pruebas

El assembler se detiene con un mensaje que indica el número de línea ante: registro desconocido o mayor que `x15`, desplazamiento sin `-allow-shifts`, mnemónico o directiva desconocidos, etiqueta o constante repetida o no definida, inmediato fuera de rango, número incorrecto de operandos, operando de memoria mal escrito, salto a dirección impar o fuera de alcance, y programa de más de 512 palabras.

Archivo	**Tests**	**Qué verifica**
`test_golden.py`	3	`blink.s`, `7seg.s` y `buttons_leds.s` dan los mismos `.hex` del repositorio base
`test_formats.py`	8	Codificaciones calculadas a mano que los ejemplos no cubren
`test_pseudo.py`	16	Pseudo-instrucciones y casos de `li`, incluida la corrección de signo
`test_directives.py`	10	`.equ`, `.word` y directivas ignoradas
`test_defines.py`	3	Opción `-D`
`test_validation.py`	28	Los errores del párrafo anterior, con casos en los límites de rango
`test_roundtrip.py`	5	Ensamblar, desensamblar con `disasm.py` y volver a ensamblar da lo mismo
Total	**73**	Se corren con `make test`

Tabla 7: Pruebas del assembler, en `assembler/tests/`.

Las pruebas contra los `.hex` del repositorio base comparan con archivos generados por otra herramienta, sin depender de nuestra lectura del manual. La prueba de ida y vuelta cubre las instrucciones que esos tres ejemplos no usan.